Trust model for the PC, phone, and watch

This document describes the current mobile-client implementation and the gaps that remain before per-device least-privilege permissions exist. It was checked by static inspection of main on October 14, 2026; it is not a physical-device test.

Summary

- Pairing requires a temporary code opened on the PC, a client-generated private key, and proof of possession of that key.
- The PC stores the public key, the client-supplied name, and authorization status in ~/.config/vladmir/mobile-devices.db. The database and its directory are created with user-restricted permissions.
- The owner approves or revokes each device identity in the desktop's Mobile and security panel.
- Sessions are temporary, bound to a device identity, and invalidated by revocation, lockdown, and server restart.
- Current limitation: authorization is broad per device. The server has no separate grants for phone and watch, nor independent scopes for agenda, history, chat, voice, telemetry, library, or programming tools.
- Pairing a watch does not currently restrict it to reminders. "Owner" is not a fine-grained permission role: some critical actions are blocked for device-authenticated clients, while several routes share the same authorization.

Implemented pairing flow

1. In Vladmir Desktop, the operator opens pairing for up to five minutes. A new code replaces the previous one; lockdown prevents pairing from opening.
2. The client creates its local cryptographic identity and sends its name, public key, code, and proof-of-possession signature. The code is single-use and expires.
3. The PC registers the identity as pending; this does not grant authenticated access to protected routes.
4. The operator checks the device in the list and manually approves or revokes it. The server identifies a device by a hash of its public key, not by its displayed name.
5. An authorized device opens a signed session. The random session token exists only in server memory, lasts five minutes by default, and is bound to the device and current security generation.
6. Authenticated requests include a signature over the method, path, body hash, timestamp, nonce, request ID, and token hash. The server rejects invalid signatures, out-of-window timestamps, reused nonces, expired sessions, unauthorized devices, and active lockdown.

The protocol binds credentials to a device and helps prevent request replay. When access leaves loopback, transport security is also required: configure TLS or an authenticated tunnel; do not expose the API directly to the internet.

Stored data

Location	Data identified in source	Notes
PC: ~/.config/vladmir/mobile-devices.db	ID derived from public key, pairing name, public key, status (pending, authorized, revoked), registration time, recent nonces, and security settings	Does not store the phone/watch private key or ephemeral session tokens. SQLite/WAL is used; file and directory permissions are restricted to the local user.
Android/Wear client	The server authentication implementation proves possession of a private key, but this document does not verify the storage or backup protections in each client	Key storage and migration in APKs remain a client-specific audit item.
Running server	In-memory session tokens for up to five minutes	Restarting the process clears sessions.

Conversation memory | Messages may be shared through server memory, dependi
ng on the chat flow | Pairing does not isolate conversations by device. Avo
id sending content you do not want shared with connected clients.

This document does not claim that all Android/Wear auxiliary data is stored
 only on-device. Confirm those details in source and in each released versi
on before making a product guarantee.

Revoke a lost or unrecognized device

1. On the PC, open Mobile and security • Vladmir.
2. Refresh the list, select the matching name/identifier, and verify the de
vice before acting.
3. Choose Revoke and confirm.
4. Revocation advances the security generation, so previous sessions for th
at registration are rejected. If the device is trusted again, pair and appr
ove it anew.
5. To temporarily block all mobile devices, enable lockdown where available
 in the UI. Lockdown rejects authentication and prevents new pairing while
active.

If the PC is lost or compromised, you cannot revoke from another device thr
ough a central account: protect the operating-system account and local data
base backup. The API does not provide a central remote recovery account.

What is authorized today

Resource | Current server behavior
Health, models, and agenda | Share the general authorization of the authent
icated client.
History | Shared; the endpoint does not filter by device.
Chat and voice/call | Available to authorized clients; conversations enter
shared memory.
Wear telemetry | Has a dedicated endpoint but uses the same authorized iden
tity; that is not an ACL scope.
Clear memory and shutdown | Requests from device-authenticated clients are
blocked and require confirmation on the PC.
Knowledge library | This authentication model has no per-device scope. Acce
ss through other flows needs a separate review.
Programming/editing tools | This mechanism grants no isolated watch role an
d does not enforce explicit per-device deny rules for every operation. Do n
ot treat pairing as a complete authorization boundary.

Gaps and improvement plan

1. Add explicit per-identity profiles and scopes with least privilege by de
fault (for example, watch: reminders and status; phone: chat and voice; PC:
 administrator).
2. Separate read and write permissions for agenda, history, library, camera
, microphone, voice, attachments, and programming tools.
3. Enforce each scope on the server for every route and operation; hiding a
 client button is not a security control.
4. Add a desktop UI to inspect and change scopes per device, showing effect
, date, and status.
5. Define retention and shared-memory isolation, with an option not to save
 phone/watch chats and to delete per-device data where possible.
6. Audit Android/Wear clients for key generation and storage, backup, migra
tion, diagnostic exports, local data, and uninstall behavior.
7. Add negative authorization tests: a watch without scope cannot read the
library, call tools, access attachments, or edit the agenda; revocation cut
s an active session; scopes do not increase after reconnect.
8. Document recovery after theft/loss, including when the PC is unavailable
.

Until these changes are implemented and tested, describe the current model
as cryptographic pairing and per-device revocation with broad authorization

, not complete resource-level permissions.

Source evidence

- src/mark/device_auth.py: pairing code, proof-of-possession validation, local registry, sessions, lockdown, and revocation.
- src/mark/mobile_api.py: request authentication and mobile/Wear routes.
- src/mark/desktop.py: device management dialog, approval, and revocation.
- tests/test_device_auth.py: approval, signature/replay, lockdown, expiry, device-bound sessions, and revocation tests. These test the backend; they do not prove secure APK storage or physical Android/Wear behavior.